React: The Component Revolution

Building UIs That Scale

From DOM Manipulation Chaos to Component Harmony

How Sarah finally conquered complex user interfaces

Previously in Sarah's Journey...

JavaScript: Freed her from low-level C programming

TypeScript: Caught many mistakes before runtime

But now: TaskMaster has grown to 50+ pages!

New Problems:

- DOM manipulation everywhere
- State scattered across files
- Duplicate code for similar UI elements
- Changes require updating multiple files

What is the state?

> State scattered across files

- **State = Memory inside a component**
- It remembers data that changes over time
- When state changes → React **re-renders** the UI

Example

```
<p id="count">Count: 0</p>
<button id="btn">Increase</button>

<script>
  let count = 0;
  const countEl = document.getElementById("count");
  const btn = document.getElementById("btn");

  btn.addEventListener("click", () => {
    count++;
    countEl.textContent = "Count: " + count;
  });
</script>
```

- Works fine: just one state (count variable) and a matching UI (`Count: 0` with id = "count").
- But what if you have 50 pages, each with changing data with states and UI?

The Breaking Point

Even worse, Sarah should manage all kinds of change and corresponding updates.

```
function updateTaskList() {
    const list = document.getElementById('taskList');
    list.innerHTML = '';
    tasks.forEach(task => {
        const li = createTaskElement(task);
        list.appendChild(li);
    });
    updateCounter();
    updateProgressBar();
    updateChart();
    refreshNotifications();
    syncWithServer();
    // ... 20 more update functions
}
```

What is React?

React is the core UI library in a large ecosystem for building interactive, component-based applications through:

Key Insight: UI as a function of state

```
UI = f(state)
```

- **Components**: Reusable UI building blocks
- **Declarative**: Describe what UI should look like
- **Reconciliation**: React compares UI descriptions and applies only the needed DOM updates
- **One-way data flow**: Predictable state management

Core React Concept: Components

Think of UI as LEGO blocks

```
// A component is just a function that returns UI
function Welcome() {
    return <h1>Hello, React!</h1>;
}

// Use it like an HTML tag
<Welcome />
```

This is JSX: JavaScript + XML (HTML) syntax

- Looks like HTML but it's JavaScript
- Gets compiled to JavaScript calls (`jsx()` in the modern transform, `React.createElement()` in the classic one)

JSX Rules

1. Return **one** root element (wrap in a `<div>` or `<>...</>`).
2. Use `className`, not `class`.
3. Close every tag: `<input />`, `<br />`.
4. Use `{ }` to put a JavaScript expression in the UI: `<h1>Hello, {name}!</h1>`
5. Component names start with a capital letter (`<TaskItem />`). Lowercase names are HTML tags.
6. Event names are camelCase: `onClick`, `onChange`.

Sarah's First Component

Before React (Imperative):

In this code, she tries to create an `<li>` with a text and button for each task.

```
function createTaskItem(task) {
    const li = document.createElement('li');
    li.className = task.completed ? 'completed' : '';

    const text = document.createElement('span');
    text.textContent = task.text;

    const button = document.createElement('button');
    button.textContent = 'Delete';
    button.onclick = () => deleteTask(task.id);

    li.appendChild(text);
    li.appendChild(button);
    return li;
}
```

This is how this JavaScript function is used to create each task list.

```
const list = document.getElementById('task-list');
tasks.forEach(task => list.appendChild(createTaskItem(task)));
```

This approach works — but becomes painful when:

- You need to update or **re-render** items
- You want to react to state changes
- You must synchronize DOM and data manually

She has to write `createElement`, `appendChild`, and `innerHTML` again and again.

After React (Declarative):

```
interface Task {
    id: number;
    text: string;
    completed: boolean;
}

interface TaskItemProps {
    task: Task;
    onDelete: (id: number) => void;
}

function TaskItem({ task, onDelete }: TaskItemProps) {
    return (
        <li className={task.completed ? 'completed' : ''}>
            <span>{task.text}</span>
            <button onClick={() => onDelete(task.id)}>
                Delete
            </button>
        </li>
    );
}
```

It is used as if it is an HTML tag.

```
<TaskItem key={task.id} task={task} onDelete={handleDelete} />
```

`key` gives React a stable, unique identity for this item among its siblings, helping React track list changes: It is used by React and is not passed to `TaskItem` as a prop.

Sarah: "I just describe what I want, not how to build it!"

Props: Component Communication

Props (properties) are **how** we pass data to components. Let's add a checkbox, so we need one more prop: `onToggle`.

```
interface TaskItemProps {
    task: Task;
    onToggle: (id: number) => void;
    onDelete: (id: number) => void;
}
```

```
function TaskItem({ task, onToggle, onDelete }: TaskItemProps) {
    return (
        <li>
            <input
                type="checkbox"
                checked={task.completed}
                onChange={() => onToggle(task.id)}
            />
            <span>{task.text}</span>
            <button onClick={() => onDelete(task.id)}>×</button>
        </li>
    );
}
```

Props are **read-only**: a component never changes its own props.

State: Component Memory

State (variable) is data that changes over time:

```
import { useState } from 'react';

function Counter() {
    // Declare state variable
    const [count, setCount] = useState(0);
    return (
        <div>
            <p>Count: {count}</p>
            <button onClick={() => setCount(count + 1)}>
                Increment
            </button>
        </div>
    );
}
```

Key: When state changes, React re-renders automatically!

The useState Hook

Instead of managing the state (count) manually, we use the `useState` function to get both the state (count) and the method to change the state (setCount).

```
const [count, setCount] = useState(0); // default value is 0

<button onClick={() => setCount(count + 1)}>
  Increment
</button>
```

Rules for State

1. Never change state directly. Create a new value.

```
// Wrong: count is state and must not be reassigned
count = count + 1;

// Right: use the setter to ask React to update state
setCount(count + 1);
```

2. When queuing multiple updates based on the previous state, use the function form.

A single `setCount(count + 1)` is fine. But multiple calls in the same event read the same `count` value from the current render:

```
setCount(count + 1);
setCount(count + 1);       // Both calls use the same count, so it increases by 1

setCount((previousCount) => previousCount + 1);
setCount((previousCount) => previousCount + 1);
// Each update uses the latest value; count increases by 2
```

React applies these updater functions in order, passing each one the latest pending state.

Rule 3: Arrays and Objects Too

The same rules apply to arrays and objects. `push` changes the **old** array, so React sees the same array and thinks nothing changed. Create a **new** array instead. Spread, `filter`, and `map` (from the JavaScript chapter) all return a new array:

```
// Wrong: changes the old array
tasks.push(newTask);
setTasks(tasks);

// Right: create a new array
setTasks([...tasks, newTask]);                    // add
setTasks(tasks.filter((t) => t.id !== id));       // delete
setTasks(tasks.map((t) =>                         // update one item
  t.id === id ? { ...t, completed: !t.completed } : t));
```

Controlled Form

The input stays synchronized with React state through `value` and `onChange`. This makes it easy to validate the text, send it to the parent on submit, and clear the field afterward.

```
const [text, setText] = useState('');
...
onAdd(text);
setText(''); // Clear input
value={text}
onChange={(e) => setText(e.target.value)}
```

```tsx
import { useState, type FormEvent } from 'react';

interface TaskInputProps {
    onAdd: (text: string) => void;
}

function TaskInput({ onAdd }: TaskInputProps) {
    const [text, setText] = useState('');

    const handleSubmit = (e: FormEvent) => {
        e.preventDefault();
        if (text.trim()) {
            onAdd(text);
            setText(''); // Clear input
        }
    };

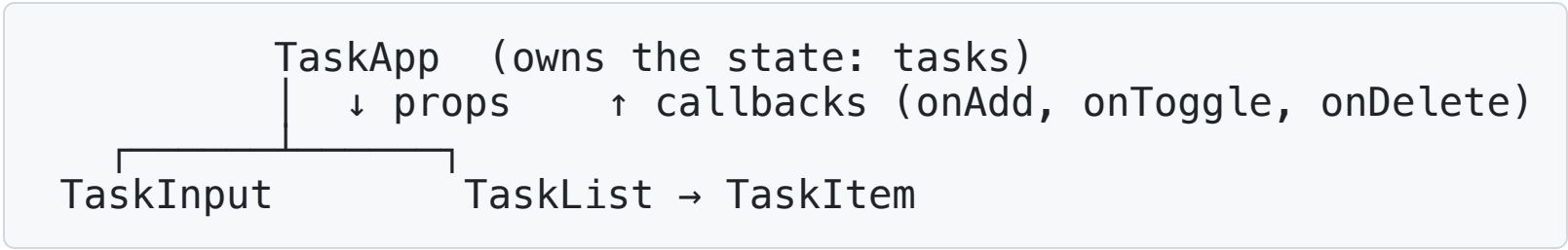
    return (
        <form onSubmit={handleSubmit}>
            <input
                value={text}
                onChange={(e) => setText(e.target.value)}
                placeholder="What needs to be done?"
            />
            <button type="submit">Add</button>
        </form>
    );
}
```

A parent uses `TaskInput` by passing a function to its `onAdd` prop. When the user submits text, `TaskInput` calls that function with the text:

```
function ExampleParent() {
    const handleAdd = (text: string) => {
        console.log('New task:', text);
    };

    return <TaskInput onAdd={handleAdd} />;
}
```

Where Should the State Live?



When several components need the same data, move the state to their closest common parent. This is called **lifting state up**.

- Data flows **down** as props.
- Events flow **up** by calling callback props.
- One owner of the data means one source of truth.

Virtual DOM and Reconciliation

Problem with direct DOM manipulation: you decide *how* to update the page, step by step.

```
// You decide every DOM update yourself
element.textContent = newContent;
element.classList.add('active');
```

React's approach:

1. React renders a lightweight description of the UI (the Virtual DOM)
2. When state changes, it renders a new description
3. Compares (diffs) old vs new
4. Applies only the necessary DOM updates

The main benefit is the declarative model. It is not automatically faster than carefully written DOM code: measure before optimizing.

Sarah Discovers useEffect

Problem: "When I refresh the page, my tasks disappear! I need to save tasks to localStorage!"

- React keeps data in **memory** (state): But once you refresh the page — **it's gone**.
- We need a **persistent place** to save data → that's where `localStorage` comes in.

What is `localStorage`?

- Built-in browser storage
- Saves data as **key-value pairs**
- Persists even after reload or browser close (until user clears it manually)

```
localStorage.setItem('name', 'Sarah');
localStorage.getItem('name'); // "Sarah"
```

Why do we need `useEffect`?

React has two worlds:

1. Rendering – What the screen looks like (pure, predictable)
2. Side Effects – Things that happen outside React (fetching, saving, subscribing)

useEffect handles the second world — anything that touches the outside world.

When React renders:

- It draws your UI based on state + props
- It should **not** change anything outside React: write to storage, call the network, change the page title, etc.

Why?

- Because React may re-render many times
- It must stay pure (no side effects during render).

The `useEffect` as the Solution

> useEffect = “Do something after render”

It tells React:

"Once you’ve updated the screen, now run this code."

Example:

```
useEffect(() => {
  console.log("App rendered!");
});
```

This runs after React finishes painting the UI — not

The Dependency Array

The second argument tells React **when** to run the effect again.

```
useEffect(() => { ... });            // after every render
useEffect(() => { ... }, []);        // once, after the first render
useEffect(() => { ... }, [tasks]);   // after renders where tasks changed
```

Rules of Hooks

`useState` and `useEffect` are called **hooks**.

- Call hooks only at the top level of a component function.
- Never call them inside `if`, loops, or nested functions.
- Hook names start with `use`.

React remembers each hook by the **order** of the calls, so the order must be the same in every render.

```
function TaskApp() {
    // Load from localStorage once, when the state is first created
    const [tasks, setTasks] = useState<Task[]>(() => {
        const saved = localStorage.getItem('tasks');
        return saved ? JSON.parse(saved) : [];
    });

    // Run side effect after render: save whenever tasks change
    useEffect(() => {
        localStorage.setItem('tasks', JSON.stringify(tasks));
    }, [tasks]); // Only run when tasks change

    // ... rest of component
}
```

`useState([])` passes an initial value directly.

- Here, `useState(() => ...)` passes a function that React calls when initializing state, not on every render.
- This avoids reading `localStorage` again on every render.

Sarah's Revelation

Sarah: "It's not just less code—it's better organized code!"

Before React:

- Lots of manual DOM manipulation
- State scattered everywhere
- Hard to add features
- Bugs from inconsistent updates

After React:

- Declarative components
- Centralized state management
- Easy to add/modify features
- Predictable updates

Vanilla JS/TS:

```
// Manual DOM updates
function updateUI() {
    element.innerHTML = '';
    data.forEach(item => {
        const el = createElement(item);
        element.appendChild(el);
    });
}

// Call everywhere data changes
updateUI();
```

React:

```
// Automatic updates (types omitted)
function List({ data }) {
    return (
        <ul>
            {data.map(item =>
                <Item key={item.id} {...item} />
            )}
        </ul>
    );
}
// React handles updates!
```

React Ecosystem

Sarah discovers a whole ecosystem:

- **React Router**: Client-side routing
- **Redux/Zustand**: Advanced state management
- **React Query**: Server state management
- **Styled Components**: CSS-in-JS
- **Next.js**: Full-stack React framework
- **React Native**: Mobile apps

```
// Example: React Router
<Routes>
    <Route path="/" element={<Home />} />
    <Route path="/grades" element={<GradeCalculator />} />
    <Route path="/tasks" element={<TaskManager />} />
</Routes>
```

Summary: The Complete Journey

```
C Language (Low-level control)
    ↓ JavaScript (High-level freedom)
        ↓ TypeScript (Type safety)
            ↓ React (Component architecture)
                ↓ Happy, productive developer!
```

Each technology solved specific problems:

- **JavaScript**: Escaped manual memory management
- **TypeScript**: Caught many errors before runtime
- **React**: Tamed UI complexity

Remember...

Think in Components

Let React Handle the DOM

State Drives Everything

Next: Read a Real React App

The Grade Calculator in React:

- Components: `GradeInput`, `GradeList`, `GradeItem`, `GradeResult`
- State in the parent: `grades` (`useState`)
- Saving with `useEffect` and `localStorage`

Judge it with the same three questions as before:

1. Can the calculation be tested without a browser?
2. Can the UI change without touching the logic?